INSIGHT COMMERCE

Proyecto Final — Henry (Data Science)

Sistema de recomendación de próxima compra (Next-Basket Recommendation) para supermercado online.

Predice y rankea los Top-10 productos más probables para el siguiente pedido de cada usuario.

Demo Streamlit: http://localhost:8501 (local) | API Swagger: http://18.217.26.71:8000/docs | MLflow: http://127.0.0.1:5000 (local)

Equipo

- Barbieri, Julián — Data Scientist
- Schanton, Sofía — Data Scientist
- Ceballos Torres, Federico — Data Scientist
- Vanegas Rojas, Juan Camilo — Data Scientist

1) Resumen Ejecutivo

Insight Commerce aborda el problema de fricción en supermercados online: los usuarios deben recordar manualmente qué volver a comprar.

El proyecto implementa un sistema de recomendación personalizado que estima la probabilidad de recompra por par usuario-producto y devuelve un Top-10 por usuario.

Se trabajó con el dataset público de Instacart, datos secuenciales reales y anonimización de origen.

La solución integra pipeline de datos, feature engineering, modelado supervisado y no supervisado, API REST, Base de datos en AWS y demo interactiva con Streamlit.

El modelo final utiliza LightGBM con optimización de hiperparámetros y validación por usuario para evitar leakage.

Además, se incorporaron prácticas de MLOps: tracking con MLflow, validación de datos, testing y monitoreo de data drift.

El enfoque prioriza robustez de datos y trazabilidad metodológica por encima de velocidad de ejecución, realizando una simulación corporativa.

2) Caso de Negocio

Contexto

Los supermercados online dependen de la memoria del cliente para armar el carrito, lo que genera abandono y ventas perdidas.

Problema

No existe un sistema inteligente que anticipe los productos del próximo pedido de cada usuario.

Objetivo

Desarrollar un recomendador personalizado que incremente el Basket Size y el Customer Lifetime Value (CLV).

KPIs del proyecto

Tipo	KPI	Definición	Estado
Principal (negocio)	Basket Size Uplift	Variación del tamaño de carrito vs. baseline sin personalización	+296% vs baseline de popularidad global
Secundario (modelo)	Recall@10	% de productos recomprados presentes en el Top-10	41.1%
Proxy (sistema)	Cobertura	% de usuarios con al menos 1 recomendación relevante	100% — cold-start global implementado

Complementario	CTR de recomendaciones	Clics en recomendaciones / impresiones	Pendiente (requiere tracking en producción)
Complementario	Conversión asistida	Compras originadas desde recomendación	Pendiente (requiere tracking en producción)

3) Dataset

Fuente: Instacart Online Grocery Basket Analysis Dataset (Kaggle).

Naturaleza: datos reales, abiertos y anonimizados.

Tablas relevantes

Tabla	Descripción
orders	Secuencia temporal de órdenes por usuario
order_products__prior	Productos incluidos en órdenes históricas
order_products__train	Productos de la siguiente orden (target)
products	Catálogo de productos
aisles	Pasillos
departments	Departamentos

Volumen

- Registros procesados: ~32M
- Tamaño estimado: ~3GB

Problemas de calidad detectados y tratamiento

Problema	Impacto	Tratamiento aplicado
Alto volumen de datos	Riesgo de memoria/costo	Muestreo por usuario y particionamiento de procesos
Nulos semánticos en features temporales	Sesgo por imputación incorrecta	Conservación selectiva de NaN y tratamiento posterior en pipeline
Riesgo de leakage entre historial y target	Sobreestimación de métricas	Separación estricta prior (features) vs train (label)
Duplicados de pares usuario-producto	Distorsión de entrenamiento	Validaciones automáticas en validate_data.py
Baja frecuencia de algunos productos	Señal ruidosa	Filtros mínimos (MIN_PRODUCT_ORDERS)

4) EDA (Análisis Exploratorio)

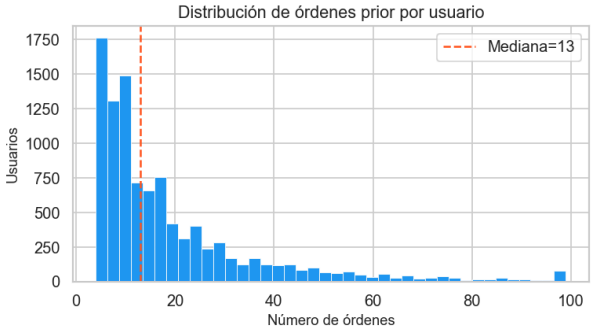
Hallazgos clave

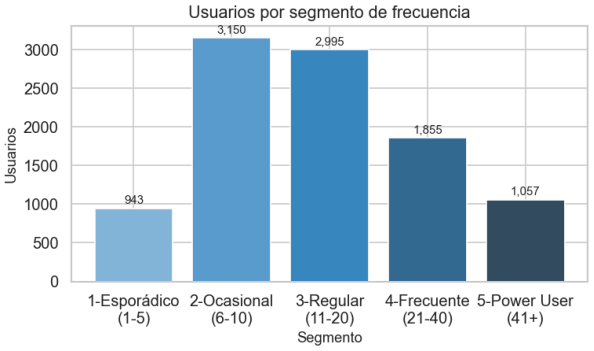
- El comportamiento de recompra presenta alta concentración en subconjuntos de productos frecuentes.
- Existen patrones de recencia y periodicidad útiles para anticipar recompra.

- La distribución de compras por usuario está sesgada (usuarios con actividad muy dispar).
- La estructura aisle/departamento aporta señal para generalización.

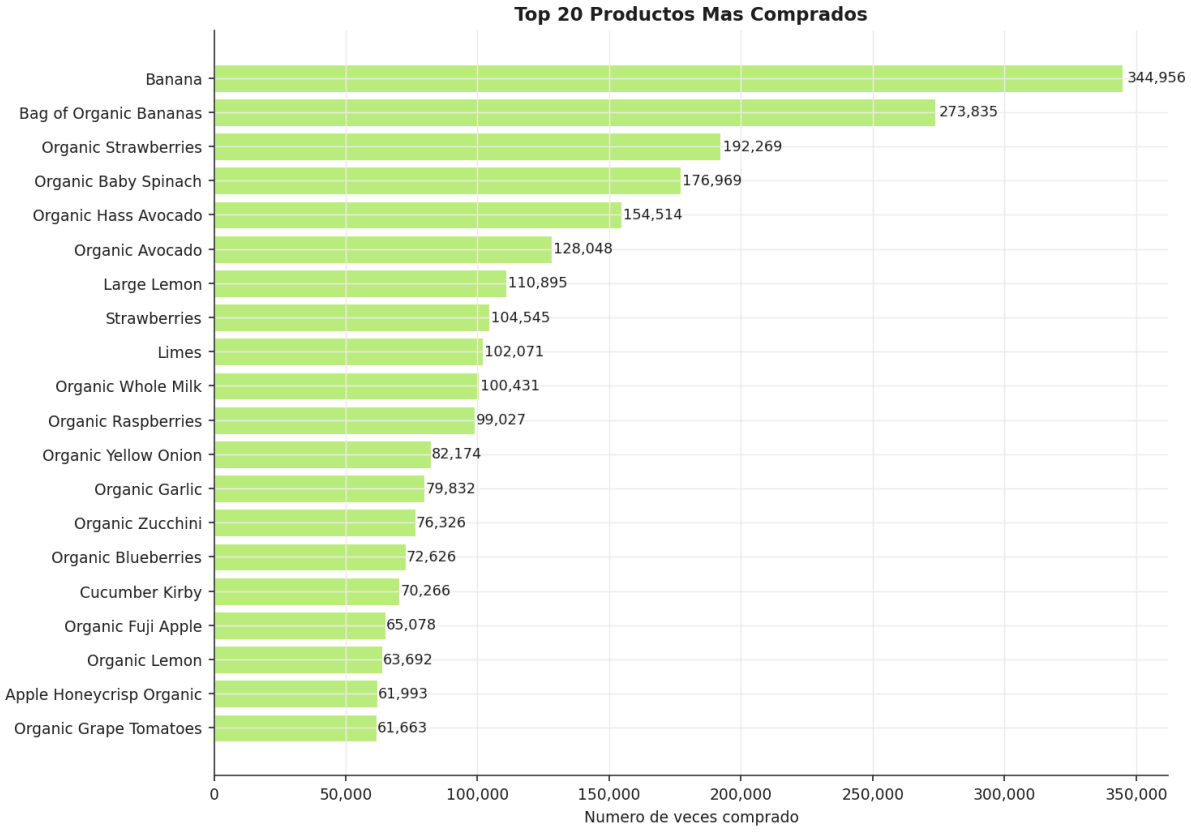
Visualizaciones destacadas

- Distribución de órdenes por segmentos

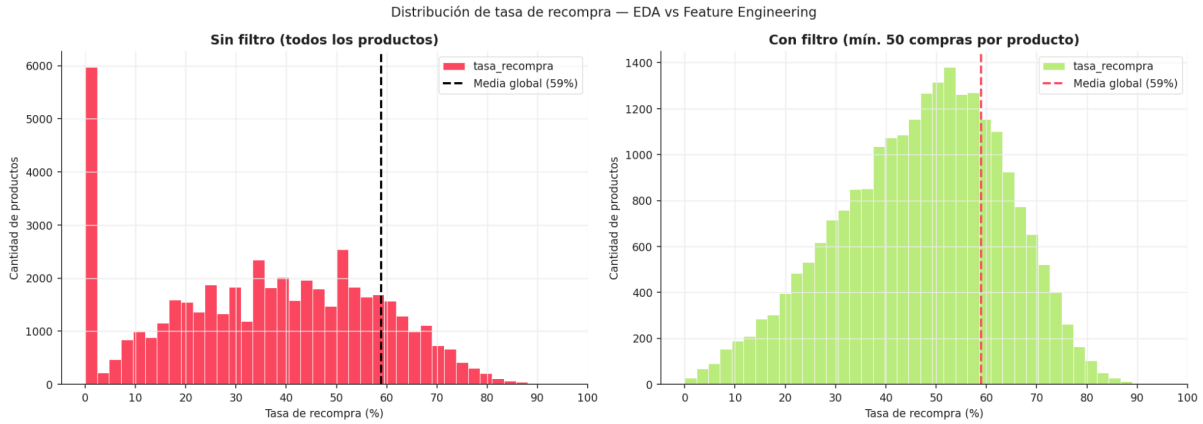




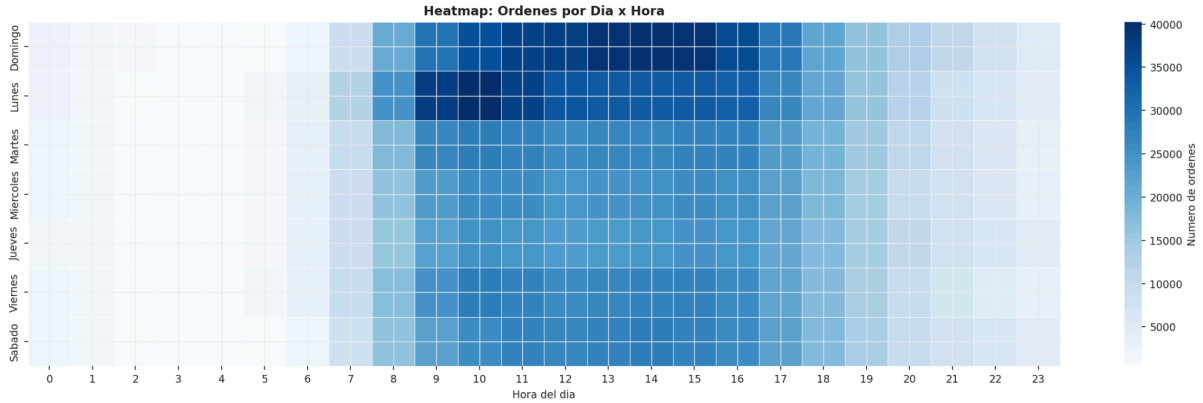
- Top 20 productos más comprados



- Distribución de tasa de recompra



- Heatmap temporal de compras



5) Feature Engineering

Se construyó una matriz de features por par (user_key, product_key) con señales de usuario, producto e interacción.

Feature	Tipo	Justificación
user_total_orders	Usuario	Captura madurez de compra y estabilidad de hábito
user_avg_basket_size	Usuario	Aproxima propensión de volumen por orden

user_days_since_last_order	Usuario	Mide recencia global del cliente
user_reorder_ratio	Usuario	Cuantifica comportamiento repetitivo vs exploratorio
user_distinct_products	Usuario	Estima diversidad de consumo
user_segment_code	Usuario	Segmentación por intensidad de compra
product_total_purchases	Producto	Popularidad global y robustez estadística
product_reorder_rate	Producto	Tendencia intrínseca de recompra
product_avg_add_to_cart	Producto	Prioridad habitual dentro del carrito
product_unique_users	Producto	Alcance real del producto
p_department_reorder_rate	Producto/Categoría	Señal jerárquica por departamento
p_aisle_reorder_rate	Producto/Categoría	Señal más granular por pasillo
up_times_purchased	Interacción	Frecuencia histórica del par usuario-producto
up_reorder_rate	Interacción	Afinidad relativa del par
up_orders_since_last_purchase	Interacción	Recencia específica del par

up_first_order_number	Interacción	Antigüedad de adopción del producto
up_last_order_number	Interacción	Último punto de contacto del par
up_avg_add_to_cart_order	Interacción	Preferencia de prioridad del par
up_days_since_last	Interacción	Tiempo desde la última recompra del par
up_avg_days_between_orders	Interacción	Ciclo medio esperado de recompra
up_delta_days	Interacción	Desfase entre ciclo esperado y observado
u_favorite_department	Usuario/Categoría	Preferencia dominante por departamento
u_favorite_aisle	Usuario/Categoría	Preferencia dominante por pasillo

Referencia técnica completa: docs/FEATURES.md

6) Modelado

Baseline

- Baseline de popularidad global (top products) para comparación.
- Estrategia simple, interpretable y útil como piso mínimo de performance.

Modelo final

- LightGBMClassifier (modelo principal).

- CatBoost utilizado como benchmark de contraste.
- Optimización de hiperparámetros con Optuna.
- Tracking de experimentos y artefactos con MLflow.

Decisiones de diseño

- Split por usuario para evitar fuga de información.
- Señales temporales + comportamentales + categóricas.
- Inclusión de clusters de usuario y producto (KMeans) como variables adicionales.
- Check de rollback por degradación de F1 antes de sobrescribir artefactos.

Esquema de validación temporal por usuario

- Entrenamiento: órdenes históricas (prior)
- Evaluación target: siguiente orden (train)
- Regla clave: prior solo para features, train solo para label.

7) Resultados

Comparativa de modelos obtenidos

Modelo	Precision	Recall	F1	AUC-ROC	NDCG@10
Baseline popularidad	0.2370	0.1159	0.1557	—	—
LightGBM baseline	0.0000	0.0000	0.0000	0.8257	—
LightGBM optimizado	0.4347	0.4106	0.4223	0.8197	0.5108

CatBoost	0.2511	0.7386	0.3748	0.8253	—

Nota: comparativa correspondiente a los resultados reportados en la Sección 12 del notebook 04_modelado.ipynb.

Métricas complementarias del modelo optimizado

- Modelo registrado: LightGBM optimizado
- Cantidad de features: 25
- Split utilizado (por usuarios): train 6,995 · val 1,499 · test 1,499
- Balanceo de clase: scale_pos_weight = 8.8837
- Top 5 variables por importancia: up_reorder_rate, up_days_since_last, product_reorder_rate, user_reorder_ratio, up_first_order_number

Indicadores reportados en Streamlit

- Precisión Top-10: 43.5%
- Poder de discriminación (AUC-ROC): 82.0%
- Cobertura de recall: 41.1%
- Usuarios evaluados (set de test): 1,499 usuarios

Evaluación sobre set de test (detalle técnico)

Métrica	Valor	Interpretación
Precision@10	0.4347	Proporción de productos relevantes en el Top-10 recomendado

Recall@10	0.4106	Proporción de productos relevantes efectivamente recuperados
F1 Score	0.4223	Media armónica entre precision y recall
AUC-ROC	0.8197	Capacidad de discriminación entre recompra y no recompra
NDCG@10	0.5108	Calidad del ranking — considera relevancia y posición de cada producto recomendado

8) Despliegue

Arquitectura del sistema

PostgreSQL (datos) → Pipeline ML (features + train + artefactos) → FastAPI (inferencia) → Streamlit (demo)

Infraestructura en AWS

- Base de datos: alojada en AWS (PostgreSQL en entorno cloud para pipeline y serving).
- API: desplegada en contenedor en AWS y expuesta vía endpoint público.

API REST (FastAPI)

- GET /health
- POST /recommend/{user_id}
- POST /recommend/batch

Estado actual: desplegada en contenedor en AWS.

Swagger productivo: http://18.217.26.71:8000/docs

Demo Streamlit

```
streamlit run app/streamlit_app.py
```

Importante: para visualizar las métricas del modelo en Streamlit, se requiere el archivo models/model_log.json (generado durante el entrenamiento).

9) Desafíos Atravesados

Restricción de costos

El volumen de 32M de registros (~3GB) superó la capacidad de almacenamiento gratuito en la nube.

Simulación corporativa

Se decidió almacenar datos crudos en la nube para replicar la arquitectura de una base de datos real de una compañía.

Migraciones de base de datos

El equipo afrontó complejidad técnica y operativa al realizar tres migraciones de base de datos durante el proyecto: Supabase → Neon → AWS. Esto implicó adaptar conexiones, credenciales, scripts de carga y validaciones de consistencia entre entornos.

Despliegue de API en AWS

También se registraron demoras durante la publicación de la API en un contenedor dentro de AWS, principalmente por ajustes de configuración de infraestructura, networking y puesta en marcha del servicio en la nube.

Compromiso técnico

Se priorizó la integridad de los datos por sobre la velocidad, resultando en ejecuciones más lentas pero con mayor flexibilidad y trazabilidad.

10) Conclusiones y Próximos Pasos

Conclusiones

- Se validó la viabilidad técnica de un recomendador Next-Basket end-to-end.
- El modelo final presenta desempeño sólido en test, con Precision=0.4347, Recall=0.4106, F1=0.4223 y AUC-ROC=0.8197, NDCG@10=0.5108.

- El modelo personalizado supera al baseline de popularidad global en un 296% de productos relevantes encontrados por usuario.
- La arquitectura implementada permite iterar mejoras sin rehacer el pipeline completo.

Próximos pasos

- Calcular y reportar KPIs de negocio en entorno real/simulado.
- Redeploy a AWS Fargate con la versión actual del código que incluye cold-start global activo para usuarios nuevos (implementado en develop, pendiente de deploy).
- Obtener mayor cantidad de métricas de los distintos modelos y compararlas entre sí.
- Implementar predicción en el Streamlit para usuarios inexistentes en el entrenamiento pero sí en la base de datos.

11) Estructura del Repositorio

```
insight-commerce-recsys/
├── app/                        # Streamlit demo
├── data/
│   ├── raw/                    # CSV originales del dataset
│   ├── processed/              # Feature matrix y derivados
│   └── local_database/         # Scripts SQL de creación de BD
├── docs/                       # Documentación técnica del proyecto
├── models/                     # Artefactos serializados del modelo
├── notebooks/                  # EDA, calidad, features y modelado
├── reports/                    # Reportes de calidad, logs y figuras
├── src/
│   ├── api/                    # FastAPI (inferencia + schemas)
│   ├── data/                   # Data loader, ETL y validaciones
│   ├── features/               # Feature engineering
│   ├── models/                 # Entrenamiento y tracking
```

```
│   ├── model_monitoring.py      # Detección de drift (PSI/KS)
│   └── pipeline.py              # Orquestador end-to-end
├── tests/                       # Tests unitarios e integración
├── requirements.txt
└── README.md
```

12) Cómo Ejecutar el Proyecto

Requisitos

- Python 3.10+
- PostgreSQL
- pip

Instalación

```
git clone https://github.com/sofiaschanton/insight-commerce-recsys.git
cd insight-commerce-recsys
python -m venv .venv
# Windows: .venv\Scripts\activate
# Linux/Mac: source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Configuración de entorno

```
# Windows: copy .env.example .env
# Linux/Mac: cp .env.example .env
```

Completar variables en .env (DB local/cloud y rutas de artefactos).

Ejecución paso a paso

1. ETL / carga dimensional: python src/data/etl_dimensional.py
2. Entrenamiento: python -m src.pipeline (o con --no-optuna para saltear Optuna)

3. API desplegada en AWS: http://18.217.26.71:8000/docs
4. Demo Streamlit: streamlit run app/streamlit_app.py
5. Tests: pytest

13) CI/CD y Aseguramiento de Calidad

El proyecto incorpora un pipeline de calidad continua con GitHub Actions y SonarCloud, orientado a detectar errores tempranos y sostener trazabilidad técnica.

Controles automáticos en GitHub Actions

En cada push y pull request (main/develop y ramas feature/fix) se ejecutan:

- Tests automáticos con pytest.
- Cobertura de código sobre src/ con reporte coverage.xml.
- Chequeo estático de tipos con mypy.
- Análisis de calidad con SonarCloud.

Además, el workflow incluye jobs de MLOps:

- Monitoreo de drift semanal y bajo demanda.
- Validación de datos antes de reentrenar.
- Reentrenamiento condicionado por drift.
- Publicación de artefactos (model.pkl, cluster_models.pkl, model_log.json).

SonarCloud

SonarCloud analiza automáticamente calidad del código en src/, test suite en tests/ y cobertura vía coverage.xml, excluyendo artefactos/no código (data/, models/, notebooks/, reports/).

Esta capa CI/CD garantiza que el modelo y la API evolucionen con validaciones reproducibles y criterios mínimos de calidad antes de integrarse.

Licencia

Este proyecto se distribuye bajo licencia MIT. Ver LICENSE.